

Software Requirements Specification (SRS)

Podcast Platform — AI-Powered Podcast Discovery & Streaming System

Table of Contents

1. [Introduction](#)
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
 2. [Overall Description](#)
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
 3. [Specific Requirements](#)
 - 3.1 External Interface Requirements
 - 3.2 Functional Requirements
 - 3.3 Non-Functional Requirements
 - 3.4 System Constraints
 4. [Use Cases](#)
 - 4.1 Use Case Diagram
 - 4.2 Detailed Use Case Descriptions
 5. [User Stories](#)
 6. [Stakeholders](#)
 7. [Appendices](#)
-
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Podcast Platform — a web-based application designed for podcast discovery, streaming, AI-powered content analysis, and user personalization. The intended audience includes the development team, project supervisor, academic evaluators, and any future maintainers of the system.

The document follows the IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998) and serves as the authoritative baseline for all design, implementation, and testing activities.

1.2 Scope

The Podcast Platform (codenamed **SonicAI**) is a client-side web application that enables users to:

- Discover podcasts through curated collections, categories, and trending algorithms.
- Search the global podcast catalog using the iTunes Search API.
- View detailed podcast and episode metadata including cover art, descriptions, publisher information, and episode lists.
- Stream audio content via a custom-built HTML5 audio player.
- Manage personal favorites with per-user localStorage persistence.
- Receive AI-generated summaries and key insights via Google Gemini API integration.
- Register, authenticate, and manage user sessions client-side.
- Track listening history, statistics, and activity patterns.
- View personalized analytics dashboards with insights derived from user behavior.

The system operates entirely within the user’s browser, leveraging third-party APIs for content and AI services while maintaining user state through browser storage mechanisms.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
AI	Artificial Intelligence — computational systems capable of performing tasks that typically require human intelligence.
API	Application Programming Interface — a set of protocols and tools for building software applications.

Term / Acronym	Definition
CSS	Cascading Style Sheets — a stylesheet language used for describing the presentation of a document.
DOM	Document Object Model — a programming interface for web documents.
Gemini	Google's generative AI model (version 2.5-flash) used for content summarization.
HTML5	HyperText Markup Language, version 5 — the standard markup language for documents designed to be displayed in a web browser.
HTTP	HyperText Transfer Protocol — the foundation of data communication for the World Wide Web.
iTunes API	Apple's iTunes Search API — a public REST API for querying podcast, music, and media metadata.
JSON	JavaScript Object Notation — a lightweight data-interchange format.
JWT	JSON Web Token — a compact, URL-safe means of representing claims to be transferred between two parties.
localStorage	A web storage mechanism that allows JavaScript sites and apps to store key-value pairs in a web browser with no expiration time.
SPA	Single-Page Application — a web application or website that interacts with

Term / Acronym	Definition
	the user by dynamically rewriting the current page rather than loading entire new pages.
UI	User Interface — the space where interactions between humans and machines occur.
UX	User Experience — a person's emotions and attitudes about using a particular product, system, or service.
Vite	A modern frontend build tool that provides a faster and leaner development experience.

1.4 References

Reference	Description
[2]	iTunes Search API Documentation, Apple Inc. — https://developer.apple.com/library/archive/documentation/AudioVideo/Conceptual/iTuneSearchAPI/
[3]	Google Gemini API Documentation — https://ai.google.dev/
[4]	React Documentation — https://react.dev/
[5]	Vite Documentation — https://vitejs.dev/

1.5 Overview

The remainder of this document is organized as follows:

- **Section 2** provides an overall description of the product, including its perspective, functions, user classes, operating environment, constraints, and assumptions.

- **Section 3** details the specific requirements, including external interfaces, functional requirements, non-functional requirements, and system constraints.
 - **Section 4** presents the use cases that describe interactions between users and the system.
 - **Section 5** captures user stories from the perspective of different user types.
 - **Section 6** identifies all project stakeholders and their interests.
 - **Section 7** contains appendices with supplementary information.
-

2. Overall Description

2.1 Product Perspective

The Podcast Platform is a standalone, self-contained web application. It does not require a proprietary backend server or database infrastructure for its core functionality. Instead, it operates as a client-side application that consumes external third-party services:

- **iTunes Search API** serves as the primary content provider for podcast metadata, episode listings, and search functionality.
- **Google Gemini API** provides AI-powered content analysis, summarization, and insight generation.
- **Browser localStorage** acts as the client-side persistence layer for user authentication state, favorites, theme preferences, listening history, and activity logs.

2.2 Product Functions

The system shall provide the following major functional groups:

F1. User Authentication & Account Management - User registration with name, email, phone, and password. - User login with email and password validation. - Automatic session persistence via localStorage. - Logout with session termination. - Remember Me functionality for email retention.

F2. Podcast Discovery & Browsing - Home dashboard with hero search, featured podcasts, categories, and trending. - Category-based browsing with iconography and color coding. - Trending podcasts sorted by popularity metrics. - Curated podcast collections.

F3. Search Functionality - Full-text search across podcast titles, publishers, and descriptions. - Search result display with loading states and empty states. - Query persistence across navigation.

F4. Podcast & Episode Details - Detailed podcast view with cover art, metadata, publisher info, and episode list. - Individual episode view with description, publish date, duration, and audio player. - Deep linking to specific podcasts and episodes.

F5. Audio Playback - Custom HTML5 audio player with play/pause controls. - Progress tracking and duration display. - Minimal player mode for embedded use in episode cards. - Audio source fallback handling.

F6. AI-Powered Content Analysis - One-click podcast summarization via Gemini API. - Per-episode summarization with structured output (Overview, Key Points, Target Audience). - Markdown rendering of AI-generated content. - Graceful error handling for API quota limits and failures.

F7. Favorites Management - Add/remove podcasts from personal favorites. - Per-user favorites persistence in localStorage. - Favorites page with grid display. - Toast notifications for add/remove actions. - Heart icon toggle on podcast cards.

F8. Theme & Personalization - Dark/light mode toggle. - System preference detection for initial theme. - Per-user theme persistence. - Smooth theme transitions.

F9. Listening History & Statistics - Track episodes played with timestamps. - Aggregate listening statistics (total time, episodes count, genres). - Activity monitoring (searches, clicks, favorites activity).

F10. User Analytics Dashboard - Personalized insights dashboard. - Visual representation of listening patterns. - Favorite genres analysis. - Activity timeline. - Recommendations based on listening history.

Constraint ID	Description	Rationale
C1	No backend server — all logic must run client-side.	Academic scope limitation; reduces infrastructure complexity.
C2	localStorage limit of ~5MB per origin.	Browser-imposed limit; data must be compressed and bounded.
C3	iTunes API rate limits — approximately 20 calls per minute without authentication.	Third-party API restriction; caching required.
C4	Gemini API free tier quota — limited requests per day.	Cost constraint; graceful degradation required.
C5	No persistent database — all user data stored in browser.	Constraint C1 consequence; data loss on browser clear.
C6	Client-side password storage — passwords stored in localStorage (hashed).	Security trade-off due to C1; users must be informed this is a demo system.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

UI-1: Home Page - Hero section with search bar, headline, and subheadline. - Stats bar displaying platform statistics (podcast count, episode count, categories, listeners). - Featured podcasts grid (6 items) with fade-in animations. - Popular categories grid (8 items) with icons and color coding. - AI Features section highlighting summarization, key takeaways, and smart search. - Trending podcasts grid (6 items) with “View All” link. - Responsive: single column on mobile, 2 columns on tablet, 4 columns on desktop.

UI-2: Search Results Page - Search query display with result count. - Podcast card grid with lazy-loaded images. - Empty state for no results. - Loading spinner during API calls. - Pagination or infinite scroll (if applicable).

UI-3: Podcast Details Page - Podcast cover art with favorite toggle button. - Title, publisher, genre badge, description. - Episode count and latest publish date. - AI Summary panel with generate button, loading state, error state, and markdown output. - Episodes list with title, date, duration, description preview, and audio player. - Responsive: stacked on mobile, side-by-side on desktop.

UI-4: Episode Details Page - Episode title, podcast link, publish date, duration. - Full description. - Custom audio player with full controls. - Related episodes or podcast link.

UI-5: Favorites Page - Header with favorites count. - Grid of favorited podcasts (same card component as search). - Empty state with call-to-action to explore. - Remove favorite capability on each card.

UI-6: Login Page - Email and password inputs with validation. - “Remember Me” checkbox. - Link to registration page. - Error messages for invalid credentials.

UI-7: Register Page - Name, email, phone, password inputs. - Password strength indicator (optional). - Link to login page. - Success toast on registration.

UI-8: User Analytics Dashboard - Summary cards (total listening time, episodes played, favorite genres). - Listening activity chart/timeline. - Recent activity list. - Personalized recommendations based on history. - Responsive grid layout.

UI-9: Navigation & Layout - Sidebar with logo, navigation links, and user section. - Top navbar with search bar, notification bell, profile icon. - Mobile hamburger menu. - Footer with links and copyright. - Player bar (if applicable) fixed at bottom.

UI-10: Theme System - Toggle button in navbar or sidebar. - Smooth transition between themes. - All components must respect theme variables.

3.1.2 Hardware Interfaces

The system does not directly interface with hardware. However, it relies on: - Standard audio output devices (speakers, headphones) for playback. - Touchscreens on mobile devices for gesture-based interaction. - Keyboard for form input and accessibility navigation.

3.1.3 Software Interfaces

3.2 Functional Requirements

3.2.1 User Authentication (AUTH)

AUTH-1: User Registration - The system shall allow a new user to register by providing: full name, email address, phone number, and password. - The system shall validate that the email address is unique (not already registered). - The system shall store the user account in localStorage within an accounts array. - The system shall hash the password before storage (basic hashing for demo purposes). - The system shall automatically log in the user upon successful registration. - The system shall display a success toast notification upon registration.

AUTH-2: User Login - The system shall allow a registered user to log in using email and password. - The system shall validate credentials against the stored accounts in localStorage. - The system shall display an error message if credentials are invalid. - The system shall create a user session upon successful login. - The system shall support “Remember Me” to pre-fill the email on subsequent visits. - The system shall automatically restore the session on app load if the user was previously logged in.

AUTH-3: User Logout - The system shall allow a logged-in user to log out. - The system shall clear the active session from localStorage. - The system shall redirect the user to the home page after logout. - The system shall not delete the user’s favorites or theme preferences on logout.

AUTH-4: Session Management - The system shall maintain the login state across browser refreshes. - The system shall store the logged-in user object in localStorage under the key user. - The system shall store a boolean flag isLoggedIn in localStorage. - The system shall check both keys on application initialization to determine login state.

46.2 Stakeholder Analysis Matrix

Influence		
High	SH-2 (Supervisor) SH-5 (Evaluators)	SH-1 (Team) SH-4 (Users)

Medium	SH-3 (University)	SH-6 (Maintainers)	
Low	SH-7 (API Providers)	SH-8 (Hosting)	
	Low	Medium	High
	Interest		

6.3 Stakeholder Requirements Summary

Stakeholder	Key Requirements
Development Team	Clear requirements, modern tech stack, manageable scope, learning opportunities
End Users	Fast, responsive, intuitive, feature-rich podcast experience
Academic Evaluators	Traceable requirements, testable features, professional presentation
Future Maintainers	Clean architecture, comprehensive docs, minimal technical debt
API Providers	Respectful API usage, proper error handling, user-agent identification
Hosting Platform	Static assets only, no server-side logic, efficient builds

7. Appendices

Appendix A: Data Model

A.1 User Account Object

```
{
  "name": "string",
  "email": "string (unique)",
  "phone": "string",
  "password": "string (hashed)"
}
```

A.2 Podcast Object (Internal)

```
{
  "id": "number",
  "title": "string",
  "publisher": "string",
  "image": "string (URL)",
}
```

```

    "description": "string",
    "total_episodes": "number",
    "latest_pub_date_ms": "number (timestamp)",
    "primaryGenreName": "string",
    "feedUrl": "string",
    "collectionViewUrl": "string"
}

```

A.3 Episode Object (Internal)

```

{
  "id": "number",
  "title": "string",
  "description": "string",
  "audio": "string (URL)",
  "audio_url": "string (URL)",
  "podcast_id": "number",
  "podcast_title": "string",
  "pub_date_ms": "number (timestamp)",
  "audio_length_sec": "number",
  "image": "string (URL)",
  "primaryGenreName": "string"
}

```

A.4 Listening History Entry

```

{
  "episodeId": "number",
  "podcastId": "number",
  "title": "string",
  "timestamp": "number (Date.now())",
  "durationListened": "number (seconds)"
}

```

A.5 localStorage Schema

Key	Type	Description
podcastAccounts	JSON Array	All registered user accounts
user	JSON Object	Currently logged-in user
isLoggedIn	String ("true"/"false")	Login state flag
podcastFavorites_ {email}	JSON Array	Favorites for specific user
podcastFavorites_ lastUser	JSON Array	Backup of last user's favorites
darkMode	String ("true"/"false")	Global theme preference
darkMode_{email}	String ("true"/"false")	Per-user theme preference
listeningHistory_ {email}	JSON Array	Listening events for user

Key	Type	Description
savedEmail	String	Remember Me email
rememberMe	String (“true”/“false”)	Remember Me flag
